

Name and Surname: Tokollo Ramabina

Student ITS No: 402306773

Qualification: BSC

Year of Study: 1

Semester: 2

Assignment due date: 30/09/2026

Date submitted: 30/09/2026

QUESTION	EXAMINER MARKS	MODERATOR MARKS	REMARKS

ASSIGNMENT INSTRUCTIONS

Please tick each box to confirm completion.

Use Times New Roman font, size 12, with 1.5 line spacing throughout the document. Apply Harvard Referencing Style for all citations and references.

For essay-style assignments, please include the following sections:

Table of Contents

Introduction

Main Body (with relevant subheadings)

Conclusion

References

Submit the assignment in PDF format on Moodle.

Use the specified cover page provided.

Include a signed declaration of originality.

DECLARATION OF ORIGINALITY:

I hereby declare that this assignment is my own work and has not been copied from any other source except where due acknowledgment is made. I affirm that all sources used have been properly cited and that this submission complies with the institution's policies on academic integrity and plagiarism.

Student Signature

A handwritten signature in black ink, consisting of a stylized 'A' followed by a horizontal line and a large, sweeping loop.

Date: 17/09/2026

Richfield Graduate Institute of Technology (Pty) Ltd is registered with the Department of Higher Education & Training as a Private Higher Education Institution under the Higher Education Act, 1997, Registration Certificate No. 2000/HE07/008

Contents

1. Introduction	2
2. System Design	3
3. Database Design and Justification.....	6
4. Technologies and Tools Used	9
5. Screenshots of the Running Application	10
6. Key Code Snippets	10
7. Challenges and Solutions	15
8. Conclusion and Reflection	16
References	18

1. Introduction

Household food waste is a large and largely invisible problem. Food is bought with good intentions, pushed to the back of a fridge or cupboard, forgotten, and thrown away when it expires. The consumer loses money, and the environment carries the cost of producing, transporting and disposing of food that was never eaten. A significant part of the problem is not that people cannot cook, but that they do not realise what they already have. Common recipe apps make this worse rather than better: they present attractive recipes that require a shopping trip, so the user either buys more food (adding to the waste problem) or closes the app.

The Smart Pantry Manager addresses this directly. It is an Android application, written in Java, that records the ingredients a user actually has at home and then suggests recipes the user can cook immediately using only those ingredients. The defining rule of the application, and its entire reason for existing, is that a recipe may only be offered as a suggestion if every single ingredient it requires is already present, in at least the required quantity. A recipe that needs five ingredients when the user has four is not shown. There is no shopping list hidden behind the suggestion, and no “almost” match presented as a match.

Food waste is only one half of the problem the application solves. The second half is decision fatigue. The question “what can I cook tonight?” is difficult precisely when time and energy are short, and the first step of any useful answer is knowing what is available. By maintaining an accurate pantry and reducing the recipe library to the recipes that are genuinely possible right now, the application turns an open-ended question into a short, actionable list.

1.1 Target users

The application is aimed at three overlapping groups:

- Households that cook at home regularly and want to reduce both food waste and grocery spend.
- Students and young adults living independently for the first time, who typically have a limited pantry, a limited budget and little experience of turning basic ingredients into meals.
- Anyone who wants a fast answer to “what can I make with what I have?” at the end of a long day.

1.2 Scope of this submission

This report accompanies the application source code and a video demonstration. Its purpose is to explain the design decisions behind the application, to document the database structure, to present evidence of the application running, and to explain the parts of the code that carry the most risk. The report is structured as follows: the system design, including the screen flow and the data model; the database design and why that database was chosen; the tools used; screenshots of every screen and core function; the key code snippets with an explanation of why each was written that

way; the problems encountered and how they were resolved; and a concluding reflection on what was learned and what would be improved with more time.

2. System Design

2.1 Architectural overview

The application uses a single host Activity (MainActivity) that owns the toolbar, the navigation drawer and a Floating Action Button, and swaps five Fragments into one container as the user moves between the main screens. Two form-style screens - adding or editing an ingredient, and viewing a recipe - are implemented as separate Activities, because they are pushed on top of the current screen and need their own back behaviour and toolbar. This split keeps the navigation model simple: the drawer switches the main working area, while Intents open screens that are about a single record.

The code is separated into layers so that each concern has one home:

- data - the model classes, the SQLite helper, the seeding data and the value-level helpers that normalise names and convert units.
- logic - the recipe matching rules, which are deliberately free of any Android dependency so they can be unit-tested on the development machine.
- adapter - the two custom RecyclerView adapters that render pantry rows and recipe rows.
- ui - the Activities and Fragments.
- util - input validation, date handling and the demo pantry used for quick testing.

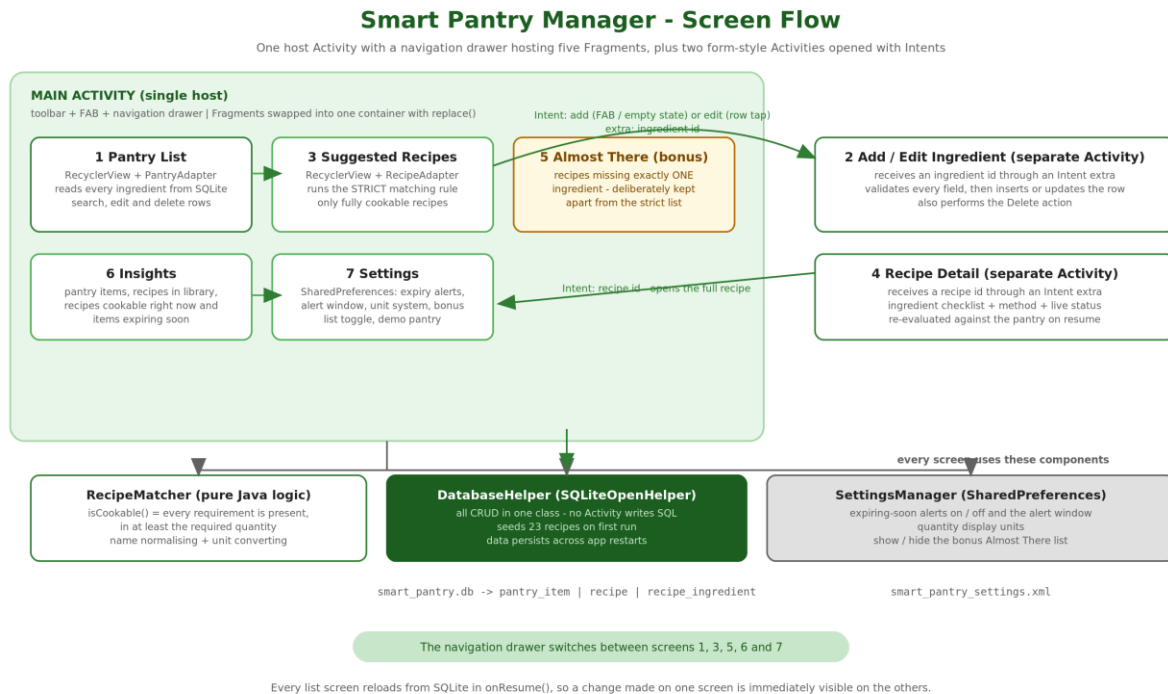


Figure 2.1 - Screen flow: one host Activity with a navigation drawer hosting five Fragments, and two form-style Activities opened with Intents.

Every list screen reloads its data from the database in `onResume()`. This is a deliberate decision: rather than caching a list in memory and keeping it synchronised with the database by hand, each screen simply re-reads the current state whenever it becomes visible. A change made on one screen is therefore visible on every other screen immediately, with no shared mutable state to keep in step. This is what makes the demonstration of the matching rule possible: deleting an ingredient on the pantry screen and returning to the suggestions screen shows the effect instantly.

2.2 Screen descriptions

Screen	Type	Purpose
1. My Pantry	Fragment	Lists every pantry item in a RecyclerView with a custom adapter. Provides live search, edit and delete, and summarises how many recipes the current pantry unlocks.

2. Add / Edit Ingredient	Activity	A single form used for both creating and updating a pantry item. Receives the record id through an Intent extra; validates every field before saving.
3. Suggested Recipes	Fragment	Runs the strict matching rule against the live pantry and lists only the recipes that can be cooked right now.
4. Recipe Detail	Activity	Shows the full ingredient checklist with per-item availability, the preparation method, and an action button that either updates the pantry or starts adding a missing item.
5. Almost There (bonus)	Fragment	Lists recipes that are exactly one ingredient short. Kept on a separate screen so a partial match can never be confused with a strict match.
6. Insights	Fragment	Summarises the pantry and the recipe library: item counts, recipes cookable now, and items expiring soon.
7. Settings	Fragment	Expiring-soon alerts and their window, quantity display units, the bonus list toggle, plus demo-pantry and clear-pantry maintenance actions.

2.3 Navigation

Navigation uses the four mechanisms the module covers. The navigation drawer switches between the five hosted Fragments. Intents move between Activities and carry data in both directions: the pantry screen sends an ingredient id so the form opens in edit mode, the suggestion lists send a recipe id so the detail screen knows which recipe to load, and the recipe detail screen sends an ingredient name to pre-fill the form when the user chooses to add a missing item. A Floating Action

Button opens the add-ingredient form from the pantry screen, and empty-state buttons route the user to the action that will fix the empty state.

2.4 The strict matching rule

The matching rule is the core business logic of the application and the piece of behaviour the assessment brief states will be tested directly. A recipe is suggested only when all of its requirements are satisfied. Two supporting pieces of logic make the rule usable in practice rather than merely correct on paper:

- Name normalisation. Pantry items typed by a human will not match recipe data exactly. Names are therefore reduced to a comparison key by lower-casing, removing punctuation and filler words such as “fresh” and “chopped”, singularising each word, and mapping known synonyms. “Tomatoes”, “tomato” and “Fresh Chopped Tomatoes” all resolve to the same key.
- Unit conversion and quantity comparison. Ingredients are compared in a base unit for their family - grams for mass, millilitres for volume, teaspoons for spoons, and items for countable goods - so that 0.5 kg of chicken in the pantry satisfies a recipe requiring 500 g, and 7 teaspoons satisfy a requirement of 2 tablespoons. A small tolerance absorbs rounding.

Where two units belong to different families and cannot be converted - for example a recipe asking for “1 tin” of tomatoes against a pantry holding 400 g - the application cannot make a reliable comparison. Rather than silently failing the recipe or silently pretending the quantities match, it accepts that the ingredient is present and displays a visible note explaining that the assumption was made. A partial match never appears in the strict suggestion list; such recipes appear only on the separate bonus screen.

3. Database Design and Justification

3.1 Choice of database

The application persists data using SQLite on the device, implemented through SQLiteOpenHelper. The assessment brief permits SQLite, Firebase or a hosted PostgreSQL database; SQLite was chosen for four reasons.

1. The data is small, structured and personal. A single user’s pantry consists of dozens of rows with a fixed shape, and the recipe library is a fixed set of reference data. This is exactly the shape of problem that relational tables solve well, and it is far too small to need a server.
2. There is no sharing, synchronisation or login. The data belongs to one person on one device. Firebase and hosted PostgreSQL would each add a network dependency and a new class of failure - poor connectivity, authentication, server uptime - without solving any problem the application actually has. A kitchen is also precisely where a network dependency is least welcome.

3. The persistence requirement is met completely and independently. SQLite stores the data in a file in the application's private storage, so the pantry survives the app being closed, force-stopped, or the device restarted. No server needs to be running for the user's data to be there.
4. It keeps the matching logic testable. Because the recipe and pantry data can be read with fast local queries, the matching algorithm can be covered by ordinary unit tests that run in seconds on the development machine, with no network stub and no emulator. Eighteen such tests accompany the project.

SharedPreferences is used for the small number of user preferences on the settings screen. These are single key–value settings with no relationships, so placing them in the relational schema would add tables and joins that model nothing. Keeping them out means the schema contains only entities that genuinely relate to one another.

3.2 Data model

The database contains three tables. The schema is created in `DatabaseHelper.onCreate()`, which runs once when the database file is first created, and the recipe tables are seeded at the same time with twenty-three recipes and their ingredient lists.

```
pantry_item(_id PK, name, quantity, unit, expiry_date) -- user-owned data (full CRUD)
recipe(_id PK, name UNIQUE, category, steps) -- reference data, seeded once
recipe_ingredient(_id PK, recipe_id FK -> recipe(_id), -- one row per recipe requirement
name, quantity, unit)
```

Listing 3.1 - Database schema.

- `pantry_item` holds the ingredients the user owns. It supports the full create, read, update and delete cycle, and carries an optional expiry date which drives the expiring-soon alerts.
- `recipe` holds the recipe itself: its name, a category, and the numbered method steps.
- `recipe_ingredient` holds one required ingredient per row, with the quantity and unit the recipe needs. `recipe_id` is a foreign key to `recipe` with `ON DELETE CASCADE`, so deleting a recipe cannot leave orphaned requirements behind. Foreign key enforcement is switched on explicitly in `DatabaseHelper.onConfigure()`, because SQLite does not enforce foreign keys by default.

There is deliberately no foreign key between the pantry and the recipes. An owned ingredient and a recipe requirement are different concepts: as soon as a recipe is cooked, the requirement is still part of the recipe, but the owned quantity has been reduced. Modelling them as separate tables and comparing them in application code keeps the recipe library stable and reusable while the pantry changes constantly.

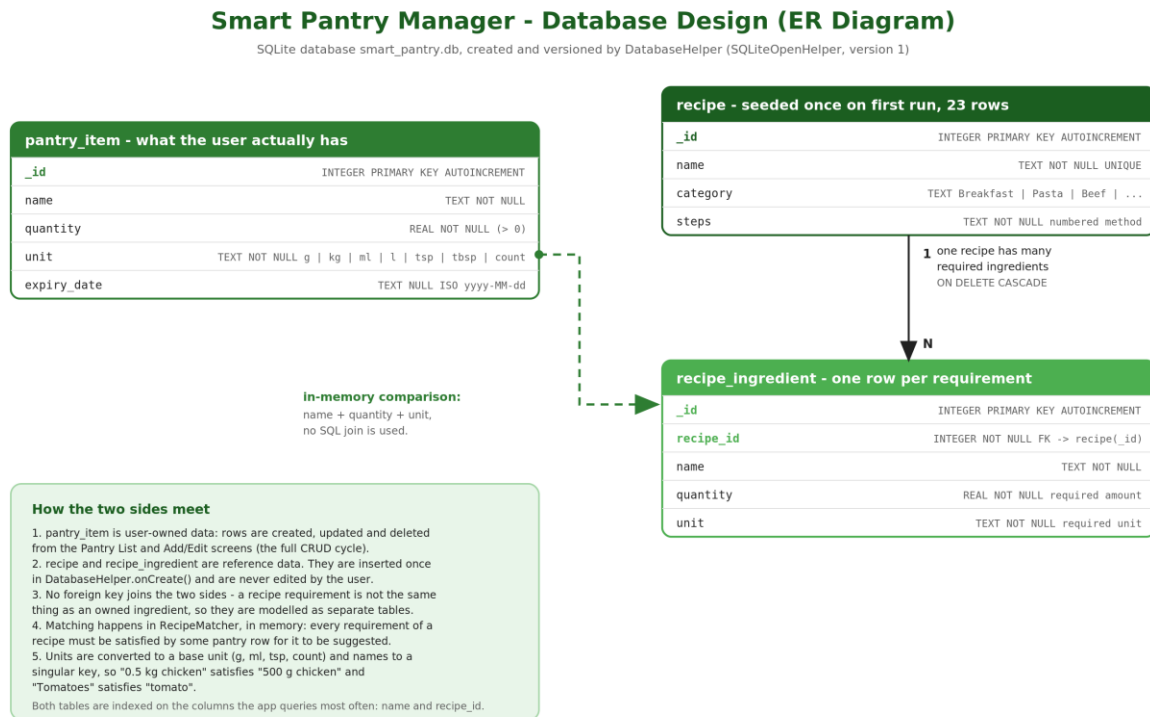


Figure 3.1 - Entity relationship diagram of the three tables and the relationship between a recipe and its required ingredients.

3.3 How the two sides meet

When a list screen loads, it reads the pantry and the recipes from the database and passes both to the matching logic. For each recipe, the matcher tests every requirement against the pantry. Testing one requirement has three steps: find pantry rows whose normalised name matches the requirement, convert the available quantity into the requirement's unit, and confirm that the available amount is at least the required amount within tolerance. If every requirement passes, the recipe is cookable; if exactly one fails, the recipe is reported as one ingredient away; if two or more fail, the recipe is not shown to the user at all. No SQL join is used for matching, because the comparison involves normalisation and unit conversion that SQL cannot express as cleanly as the Java code can.

3.4 Data integrity and indexing

- NOT NULL constraints are applied to every column the application logic depends on, so a row can never be stored without a name, quantity or unit.
- The pantry table is indexed on name, and recipe_ingredient on recipe_id, because those are the columns the application queries most often.
- Seeding runs inside a single transaction, so the first launch is either fully seeded or not seeded at all - an interrupted first run cannot leave a partial recipe library.
- The database is versioned through DATABASE_VERSION, with onUpgrade() handling schema changes in later versions. Automatic backup rules are declared in the manifest so the pantry and settings survive a device change.

4. Technologies and Tools Used

Technology	How it is used in this project
Java (Android SDK 34)	All application code. No Kotlin is used anywhere in the project.
Android Studio	The development environment: project structure, emulator, debugger and Logcat.
SQLite via SQLiteOpenHelper	On-device persistent storage for the pantry and the recipe library.
SharedPreferences	Storage for the handful of user preferences set on the settings screen.
RecyclerView and ListView-equivalent adapters	Two custom RecyclerView adapters render the dynamic lists of pantry items and recipes.
Material 3 components	Toolbar, navigation drawer, text fields, switches, cards and the Floating Action Button, with a light and a dark theme.

Gradle 8.7 / Android Gradle Plugin 8.5.2	Build configuration and dependency management.
JUnit 4	Eighteen unit tests covering the matching rule, unit conversion and name normalisation.
Git and GitHub	Version control, with incremental commits made throughout development.
Draw.io-style diagrams	The screen flow and ER diagrams included in sections 2 and 3.

5. Screenshots of the Running Application

6. Key Code Snippets

Five snippets have been selected. They are the parts of the system that carry the most risk or the most design thought, rather than a sample of the codebase.

6.1 The strict matching rule

```

public static boolean isCookable(Recipe recipe, List<Ingredient> pantry) {
    if (pantry == null || pantry.isEmpty()) {
        return false; // Nothing in the pantry means nothing can be cooked.
    }
    for (RecipeIngredient required : recipe.getIngredients()) {
        if (!IngredientMatcher.compare(required, pantry).satisfied) {
            return false; // One short ingredient is enough to exclude the recipe.
        }
    }
    return true;
}

```

Listing 6.1 - RecipeMatcher.isCookable(), the rule the application is built around.

The method is written as a loop with a single early exit, and it returns true only if it reaches the end. That structure was chosen deliberately for two reasons. First, it makes the rule obviously strict: there is no partial-scoring, no threshold and no tolerance for a missing ingredient, so the behaviour matches the specification exactly and can be read at a glance. Second, returning as soon as a requirement fails is cheaper than testing the rest of the recipe, because every requirement costs a name normalisation and possibly a unit conversion. The null and empty checks at the top guard the common case of a new user who has not added anything yet, and they mean the method can never throw a null pointer exception from the UI.

6.2 First-run seeding inside a transaction

```

private void seedRecipes(SQLiteDatabase db) {
    int recipeCount = 0, ingredientCount = 0;
    db.beginTransaction();
    try {
        for (Recipe recipe : SeedData.recipes()) {

```

```

ContentValues recipeValues = new ContentValues();
recipeValues.put(COL_NAME, recipe.getName());
recipeValues.put(COL_CATEGORY, recipe.getCategory());
recipeValues.put(COL_STEPS, recipe.getSteps());
long recipeId = db.insert(TABLE_RECIPE, null, recipeValues);
if (recipeId == -1) { continue; }
recipeCount++;
for (RecipeIngredient ingredient : recipe.getIngredients()) {
    ContentValues ingredientValues = new ContentValues();
    ingredientValues.put(COL_RECIPE_ID, recipeId);
    ingredientValues.put(COL_NAME, ingredient.getName());
    ingredientValues.put(COL_QUANTITY, ingredient.getQuantity());
    ingredientValues.put(COL_UNIT, UnitConverter.canonicalUnit(ingredient.getUnit()));
    if (db.insert(TABLE_RECIPE_INGREDIENT, null, ingredientValues) != -1) {
        ingredientCount++;
    }
}
}
db.setTransactionSuccessful();
} finally {
    db.endTransaction();
}
}

```

Listing 6.2 - Seeding the recipe library on first run as a single transaction.

Roughly one hundred and thirty rows are inserted when the database is first created. Wrapping the loop in `beginTransaction()` and `setTransactionSuccessful()` means those inserts are committed as one unit: it is considerably faster than committing each row separately, and it is all-or-nothing, so an interrupted first launch cannot leave the application with a partially seeded recipe library. The `endTransaction()` call sits in a finally block so that the transaction is always closed, even if an insert throws - which also guarantees a rollback in that case, because the transaction was never marked successful.

6.3 Unit conversion for quantity comparison

```
/** Converts a quantity into the base unit of its family (g, ml, tsp or count). */
public static double toBaseUnit(double quantity, String unit) {
    Double factor = TO_BASE.get(canonicalUnit(unit));
    return factor == null ? quantity : quantity * factor;
}

/** Converts a quantity from one unit into another unit of the same family. */
public static double convert(double quantity, String fromUnit, String toUnit) {
    if (!isConvertible(fromUnit, toUnit)) {
        return quantity; // Not convertible: caller must treat the comparison as an assumption.
    }
    double base = toBaseUnit(quantity, fromUnit);
    Double targetFactor = TO_BASE.get(canonicalUnit(toUnit));
    return targetFactor == null || targetFactor == 0d ? base : base / targetFactor;
}
```

Listing 6.3 - UnitConverter, which makes "0.5 kg" and "500 g" comparable.

Without this class the strict rule would be correct but practically useless, because a user who records chicken as 0.5 kg would never match a recipe written in grams. Every unit is mapped to a canonical spelling and a conversion factor into its family's base unit, so converting between any two units of the same family is a multiply and a divide. The `isConvertible()` guard matters as much as the arithmetic: it prevents the code from silently producing a meaningless number when two units belong to different families, and it lets the caller decide to report an assumption instead.

6.4 Binding data to the pantry list

```
@Override

public void onBindViewHolder(@NonNull PantryViewHolder holder, int position) {

    holder.bind(items.get(position));

}

void bind(final Ingredient ingredient) {

    textName.setText(ingredient.getName());

    textQuantity.setText(ingredient.getQuantityLabel());

    boolean expiringSoon = expiryAlertsEnabled && DateUtils

        .isExpiringWithin(ingredient.getExpiryDate(), expiryWindowDays);

    imageExpiryWarning.setVisibility(expiringSoon ? View.VISIBLE : View.GONE);

    buttonEdit.setOnClickListener(v -> listener.onEditIngredient(ingredient));

    buttonDelete.setOnClickListener(v -> listener.onDeleteIngredient(ingredient));

}
```

Listing 6.4 - PantryAdapter: a custom RecyclerView adapter with expiry awareness.

The adapter is the only place that decides how a pantry item looks, so the expiry rule is implemented once and applies to every row. The expiry logic reads the user's preference from the settings manager rather than hard-coding a window, which is what allows the settings screen to change the behaviour of the list without the list knowing anything about settings. Row actions are reported back through a listener interface implemented by the hosting Fragment, so the adapter never starts Activities or touches the database itself - it stays a rendering class with a single responsibility, and it can be reused on any screen that needs to show ingredients.

6.5 Validating before persisting

```
String nameError = ValidationUtils.validateName(this, name);

String quantityError = ValidationUtils.validateQuantity(this, quantityText);

String unitError = ValidationUtils.validateUnit(this, unitLabel);
```



```

String expiryError = ValidationUtils.validateExpiryDate(this, expiry);

layoutName.setError(nameError);

layoutQuantity.setError(quantityError);

layoutUnit.setError(unitError);

layoutExpiry.setError(expiryError);

if (nameError != null) { inputName.requestFocus(); return; }

if (quantityError != null) { inputQuantity.requestFocus(); return; }

// ... only once every field is valid:

databaseHelper.addIngredient(editingIngredient);

```

Listing 6.5 - Validation runs completely, then reports, then persists.

Every rule is evaluated before any error is displayed, so a user who makes two mistakes sees both at once instead of fixing them one at a time. The rules themselves live in a separate `ValidationUtils` class rather than inside the Activity, which keeps the Activity focused on screen behaviour and makes the validation rules reviewable in one place; each rule returns either null or a ready-to-display message, so the Activity does not construct error strings. Persistence is only reached after every check has passed, and focus is moved to the first offending field so the user is taken straight to the problem.

7. Challenges and Solutions

7.1 Matching messy, human-entered names without over-matching

The first version of the matching logic compared names exactly, and it failed immediately in practice: a pantry containing “Tomatoes” did not match a recipe requiring “tomato”, so the user was told they could not cook a meal they could obviously cook. The fix was a normaliser that reduces both sides of the comparison to a common key by lower-casing, removing punctuation and filler words, singularising words, and mapping known synonyms such as aubergine and brinjal.

That fix introduced a subtler and more dangerous bug. Because the normaliser removes words such as “fresh”, the phrase “fresh chilli” reduced to the same key as “chilli”, and because “chilli flakes” was listed as a synonym of chilli, a fresh chilli in the pantry was treated as satisfying a recipe that specifically required chilli flakes. Over-matching is worse than under-matching here, because it tells the user they can cook something they cannot. The solution was twofold: chilli flakes were given their own synonym group, separate from fresh chilli, and the synonym table was

made order-safe by inserting with `putIfAbsent()` so that an alias can never re-point a key that a higher-priority group has already claimed. A unit test was added specifically to keep chilli and chilli flakes distinct, which is how the regression would be caught if the synonym table is ever edited again.

7.2 Comparing quantities expressed in different units

Users record quantities in whatever unit makes sense to them - half a kilogram of chicken, a litre of milk, a tablespoon of oil - while recipes are written with whatever unit was natural to the recipe. Comparing those numbers directly gives absurd results: a pantry containing 0.5 would fail a requirement of 500, and a pantry containing 7 would fail a requirement of 2. The solution was to introduce `UnitConverter`, which maps every unit to a canonical spelling and a factor into a base unit for its family, so all comparisons happen in grams, millilitres, teaspoons or items. A tolerance of two percent (with a floor of one base unit) absorbs the rounding that comes from users typing approximate amounts, so 0.9 litres still satisfies a requirement of one litre.

The harder problem was the case where conversion is impossible. A recipe asking for “1 tin” of tomatoes cannot be compared with a pantry holding tomatoes by weight, and no amount of arithmetic will fix that. Three options were considered: fail the recipe, pass the recipe, or pass it and say so. Failing it is user-hostile, because the user almost certainly does have the ingredient. Passing it silently would make the application dishonest about its own rule. The third option was implemented: the comparison reports that it assumed the ingredient is available, and the recipe detail screen displays that note to the user. This keeps the strict rule honest - nothing is claimed that has not either been verified or been explicitly flagged.

7.3 Keeping the lists in step with the database

An early version of the pantry screen kept its list of ingredients in memory and updated it locally after each add, edit or delete. This worked until the same data was changed from another screen: editing an ingredient and returning to the pantry showed stale values, and the recipe suggestions were calculated from a pantry that no longer existed. Caching a copy of the database in the UI created two sources of truth that had to be kept manually in step, and every new screen added another opportunity for them to diverge.

The solution was to remove the cache entirely. Every list screen re-reads from the database in `onResume()`, so the displayed data is always the stored data, and a change made anywhere is visible everywhere the moment the screen becomes visible. For an application of this size the cost is negligible - a few dozen rows from a local file - and it removed an entire category of bug. The same decision is what makes the strict rule demonstrable: deleting one ingredient and returning to the suggestions screen shows the true effect immediately, with no need to force a refresh.

8. Conclusion and Reflection

8.1 What the application achieves

The completed application meets the requirements of the brief. It is written entirely in Java; it provides seven screens covering pantry management, a suggestion list, recipe detail, insights and settings; it uses Intents to navigate and to pass data between screens; it uses two custom RecyclerView adapters to display dynamic data; it validates all data entry; and it persists everything in a local SQLite database with full create, read, update and delete support that survives the app being closed. The strict matching rule - the core value of the project - is implemented in one clearly identifiable method, is supported by name normalisation and unit conversion so that it works with realistically messy input, and is covered by eighteen unit tests. No mapping, location or payment functionality is present, in line with the brief.

8.2 What was learned

- That the strict rule is the easy part. The difficult engineering is in the assumptions around it: deciding what counts as the same ingredient, what counts as enough of it, and what to do when the comparison cannot be made at all. Most of the design effort went into those questions rather than into the loop that enforces the rule.
- That over-matching is more damaging than under-matching. A user who is told they cannot cook something they could cook is mildly annoyed; a user who is told they can cook something and then discovers a missing ingredient stops trusting the application. Normalising and synonym handling therefore need to be deliberately conservative, and the temptation to be generous is a trap.
- The value of separating pure logic from Android code. Because the matching rules have no Android dependencies, eighteen tests covering the trickiest behaviour in the application run in seconds on the development machine instead of requiring an emulator, which made it far easier to iterate on the matching rules safely.
- The practical shape of the Activity and Fragment lifecycle. Reloading from the database in onResume() rather than onCreate() turned out to solve both a lifecycle question and a data-consistency problem at the same time.
- That user-facing honesty is a design decision. Deciding to display the assumed unit comparison instead of hiding it made the application more trustworthy and, in the end, made the limitation of the algorithm explainable rather than embarrassing.

8.3 What would be improved with more time

- Portion scaling, so a recipe can be adjusted to the quantity the user actually has rather than only reporting whether they have enough.
- A shopping list generated from the Almost There screen, so that the recipes the user is closest to cooking can be unlocked with the smallest possible purchase.

- Expiry notifications through WorkManager, so the alert is delivered when it is useful rather than only shown when the app is open.
- A larger and richer recipe library, with images, preparation and cooking times, and nutritional information.
- Migration to the Room persistence library, which would give compile-time verification of the SQL queries and reduce the amount of hand-written cursor-mapping code.
- An instrumented test suite covering the database layer itself, since the current tests deliberately cover only the parts of the system that contain no Android dependencies.

8.4 Closing remark

Building this application showed that a small, well-chosen rule can carry an entire product, provided the surrounding engineering is honest about the real world the rule has to operate in. The strict matching rule is four lines of Java; making it useful to a real person in a real kitchen required normalising how people name things, converting how they measure things, admitting when a comparison could not be made, and keeping the interface truthful about all three.

References

Android Developers (2024a) Save data using SQLite. Available at: <https://developer.android.com/training/data-storage/sqlite> (Accessed: September 2026).

Android Developers (2024b) SQLiteOpenHelper. Available at: <https://developer.android.com/reference/android/database/sqlite/SQLiteOpenHelper> (Accessed: September 2026).

Android Developers (2024c) Create dynamic lists with RecyclerView. Available at: <https://developer.android.com/develop/ui/views/layout/recyclerview> (Accessed: September 2026).

Android Developers (2024d) Fragments. Available at: <https://developer.android.com/guide/fragments> (Accessed: September 2026).

Android Developers (2024e) The activity lifecycle. Available at: <https://developer.android.com/guide/components/activities/activity-lifecycle> (Accessed: September 2026).

Android Developers (2024f) Intents and intent filters. Available at: <https://developer.android.com/guide/components/intents-filters> (Accessed: September 2026).

Android Developers (2024g) Save key-value data (SharedPreferences). Available at: <https://developer.android.com/training/data-storage/shared-preferences> (Accessed: September 2026).

Android Developers (2024h) Create a navigation drawer. Available at: <https://developer.android.com/develop/ui/views/components/appbar/navigation-drawer> (Accessed: September 2026).

Android Developers (2024i) Text fields (TextInputLayout). Available at: <https://developer.android.com/develop/ui/views/components/textfields> (Accessed: September 2026).

Android Developers (2024j) App architecture guide: data layer. Available at: <https://developer.android.com/topic/architecture/data-layer> (Accessed: September 2026).

Google (2024) Android Gradle plugin release notes. Available at: <https://developer.android.com/build/releases/gradle-plugin> (Accessed: September 2026).

Iversen, J. and Eierman, M. (2020) Mobile app development for iOS and Android. 3rd edn. Prospect Press.

Material Design (2024) Material 3 components and theming. Available at: <https://m3.material.io/components> (Accessed: September 2026).

Meier, R. and Lake, I. (2018) Professional Android. 4th edn. Indianapolis: Wrox.

Richfield Graduate Institute of Technology (2024) Mobile App Development 700: student guide. Johannesburg: Richfield Graduate Institute of Technology.

SQLite Consortium (2024) SQLite documentation: foreign key support. Available at: <https://www.sqlite.org/foreignkeys.html> (Accessed: September 2026).